

Resenha Design by Contract

de Bertrand Meyer

O artigo de Bertrand se baseia na construção de sistemas robustos, confiáveis e reutilizáveis, podendo ser escalados futuramente ou refatorados. A tese central do artigo se baseia em como garantir que diferentes componentes de um software interajam de forma previsível e sem erros ocultos. Bertrand propõe que a relação entre provedor e o cliente deve ser vista como um contrato formal. Um contrato comercial, ambas as partes possuem deveres e obrigações tanto quanto benefícios. Bertrand traduz isso para os softwares através de alguns elementos, as pre condições que são o que o cliente deve exigir antes de iniciar o ciclo, pos condições são o que o provedor garante que seja verdade após a execução e as invariantes de classe que são condições que devem permanecer verdadeiras durante toda a vida do objeto. Essa abordagem inverte a lógica comum da programação defensiva, onde cada função verifica exaustivamente se os dados de entrada são válidos. No Design by Contract, caso o cliente do sistema viola uma pre condição, a falha é do mesmo, e o sistema deve interromper a execução para evitar os erros em cadeia.

É argumentado sobre a maioria dos erros de software que surgem de mal entendidos sobre as responsabilidades de cada módulo do sistema. Ao normalizar as expectativas, o Design by Contract oferece uma solução para este problema através de uma sólida documentação onde o contrato não é apenas um simples comentário de texto que com o tempo pode ficar obsoleto e não necessário. A documentação do modelo é extraída diretamente das pre e pos condições, além disso é introduzido uma mecânica de ao ocorrer a quebra do contrato automaticamente já aponta para quem causou a quebra. Meyer introduziu o sistema princípio de que subclasses podem enfraquecer e fortalecer relacionamentos garantindo que uma subclasse sempre possa substituir sua superclasse sem quebrar o contrato estabelecido trazendo segurança ao sistema.

O artigo traz algo que amplamente utilizado nos dias de hoje, não é apenas uma contribuição de sintaxe ou tecnologia genérica desnecessária. Meyer traz o artigo para nos mostrar uma forma de evoluir na arquitetura de software de forma simples e direta, com simples acordos e trocas contratuais entre partes já existentes do sistema. A mudança pode prevenir extensas linhas de código com tratamento de erros e exceções com apenas trocas contratuais do sistema.